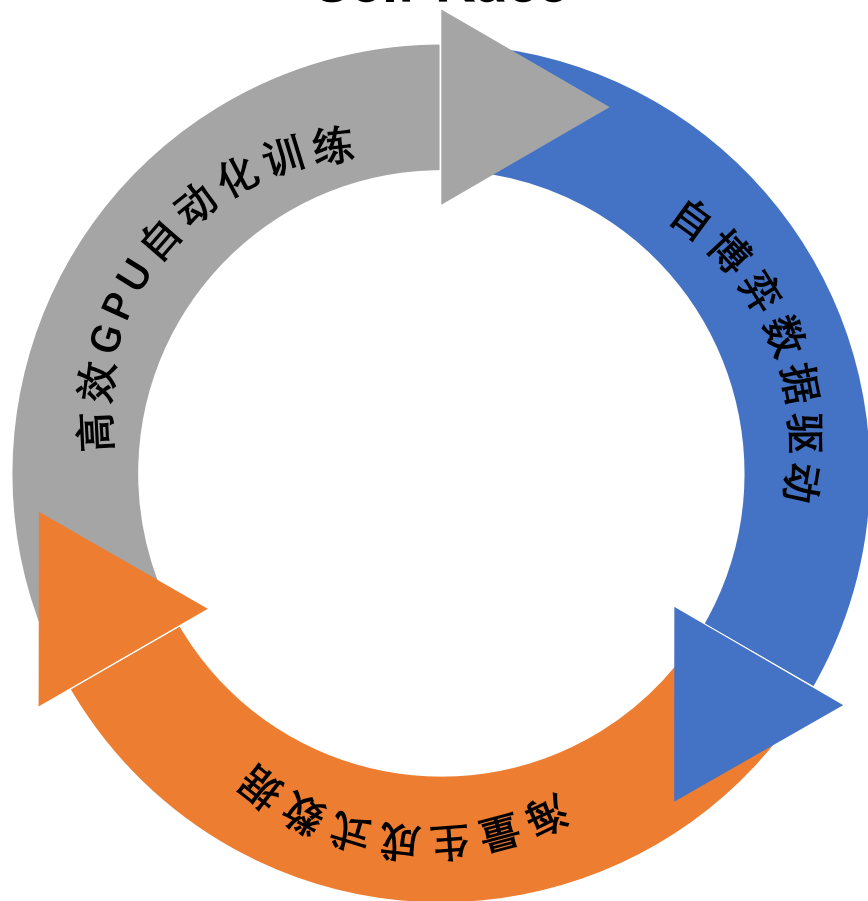


SELF  **RACE**

自博弈强化学习端到端

Self-Race



自博弈数据驱动

相比人类数据，全流程的Self-race模拟器数据驱动
的算法可实现低成本，高速的自动驾驶迭代

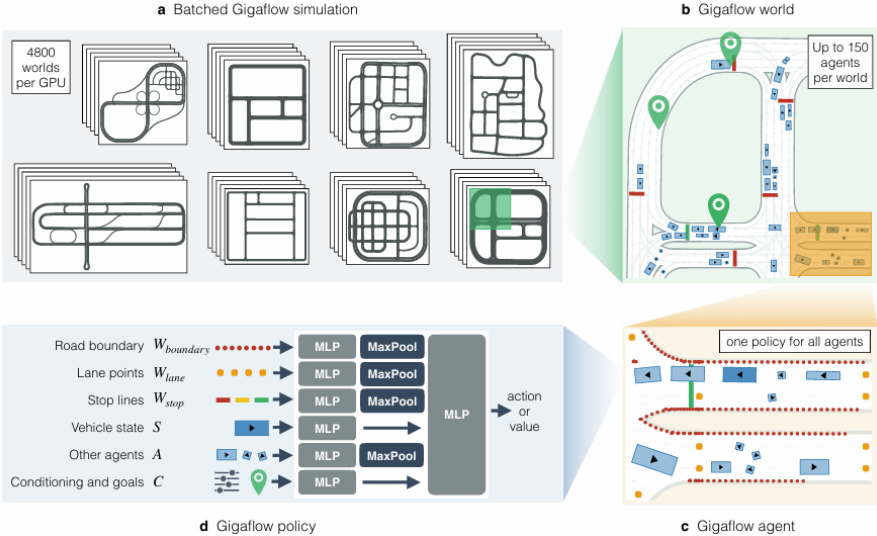
海量数据

通过8-GPU节点实现低成本(5 美元 / 百万公里)海
量数据回流。**(10天训练可以产生9500年驾驶经验)**

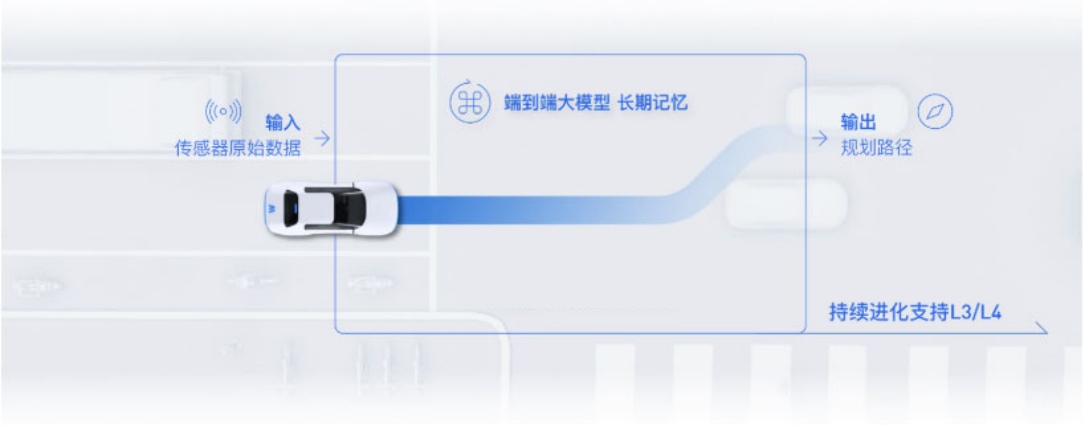
高效GPU自动化训练

低成本处理海量数据的高效自动化迭代的工具链
(基于Cuda 编程加速以及高效pytorch框架)

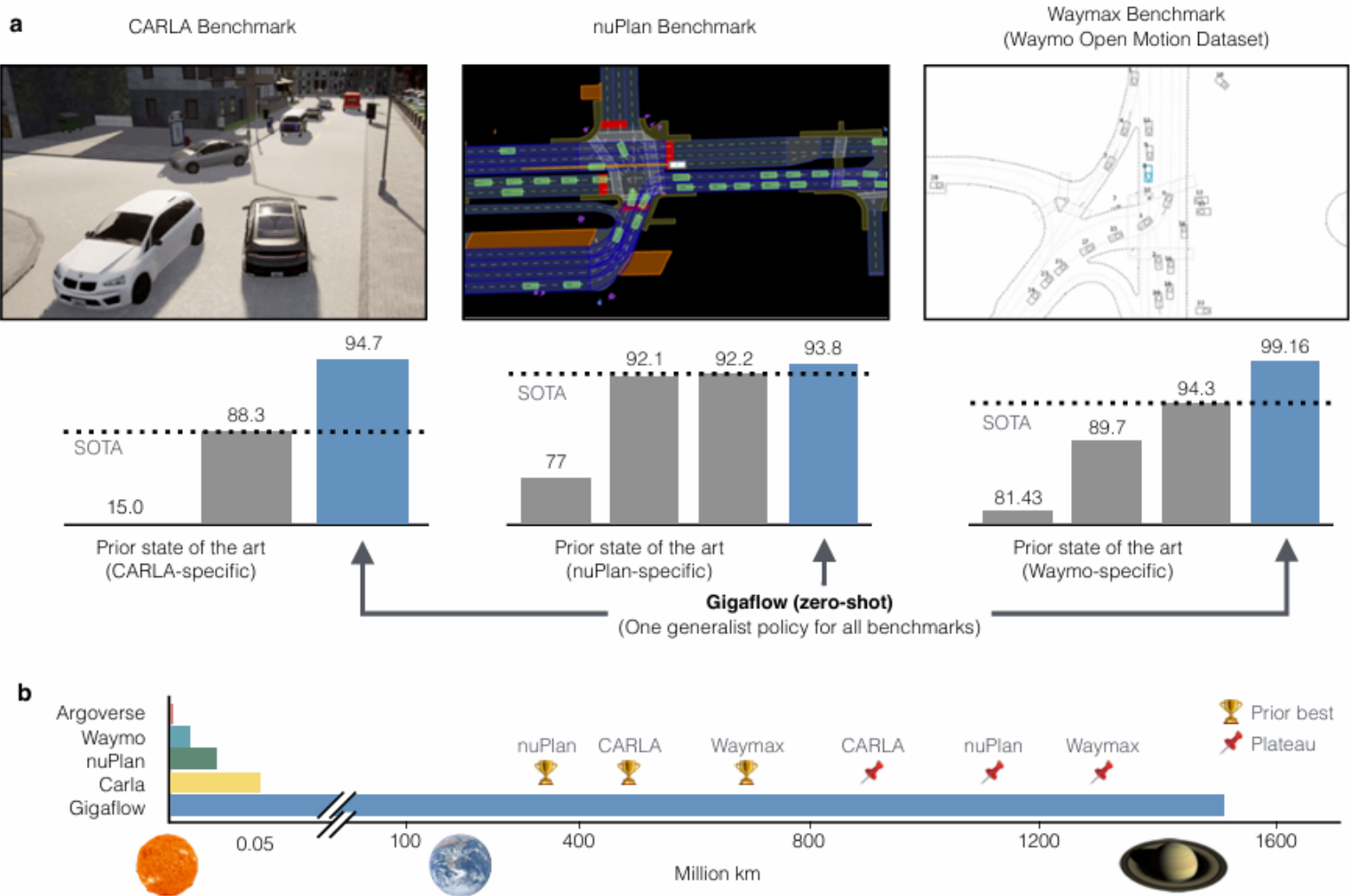
Duffytec 端到端模型



感知与规划整合进一个大模型中，模仿人类的**驾驶直觉**。同时保留MLP模型，以此保证高质量的大模型训练数据+低成本训练方法。

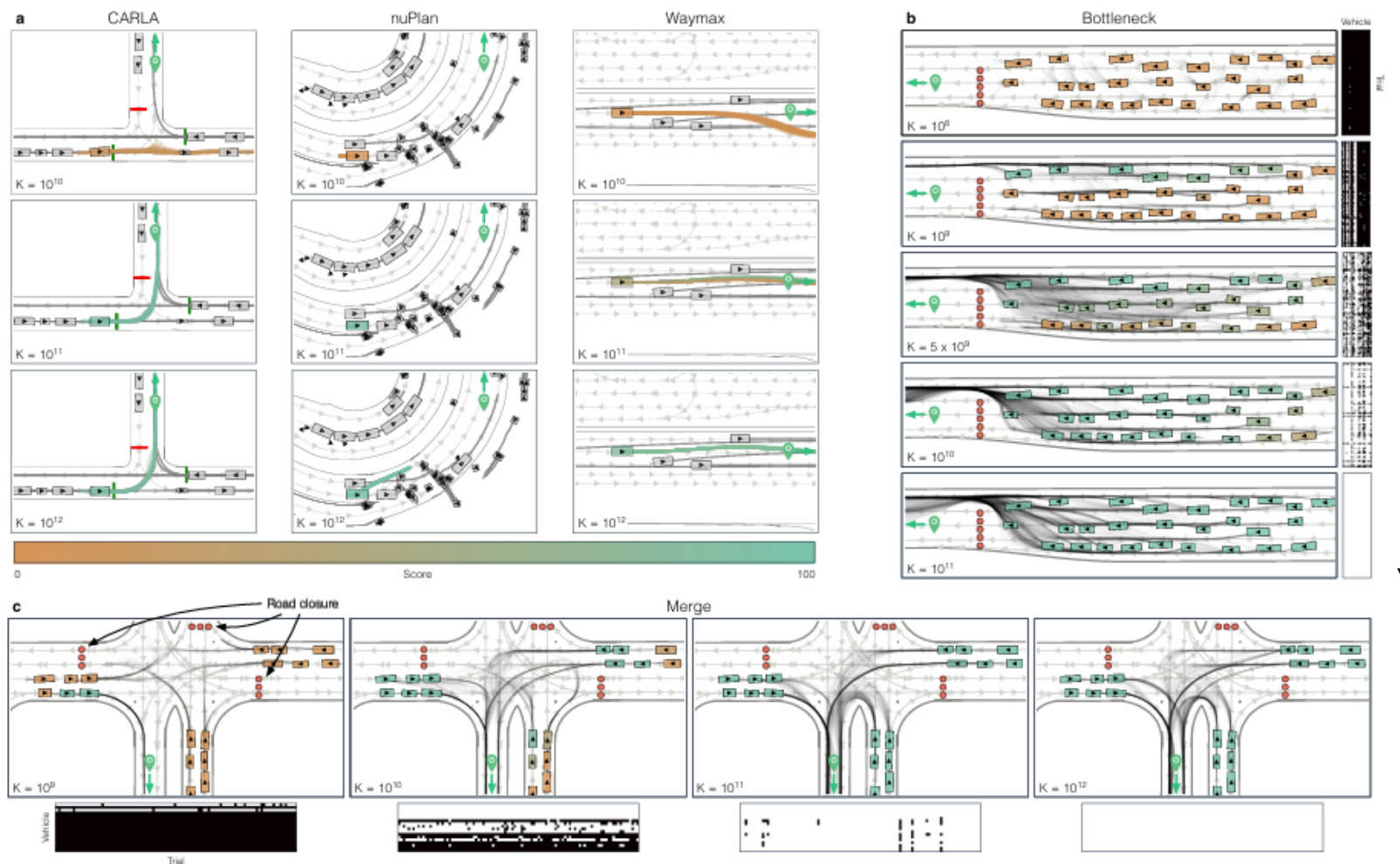


模拟测试结果



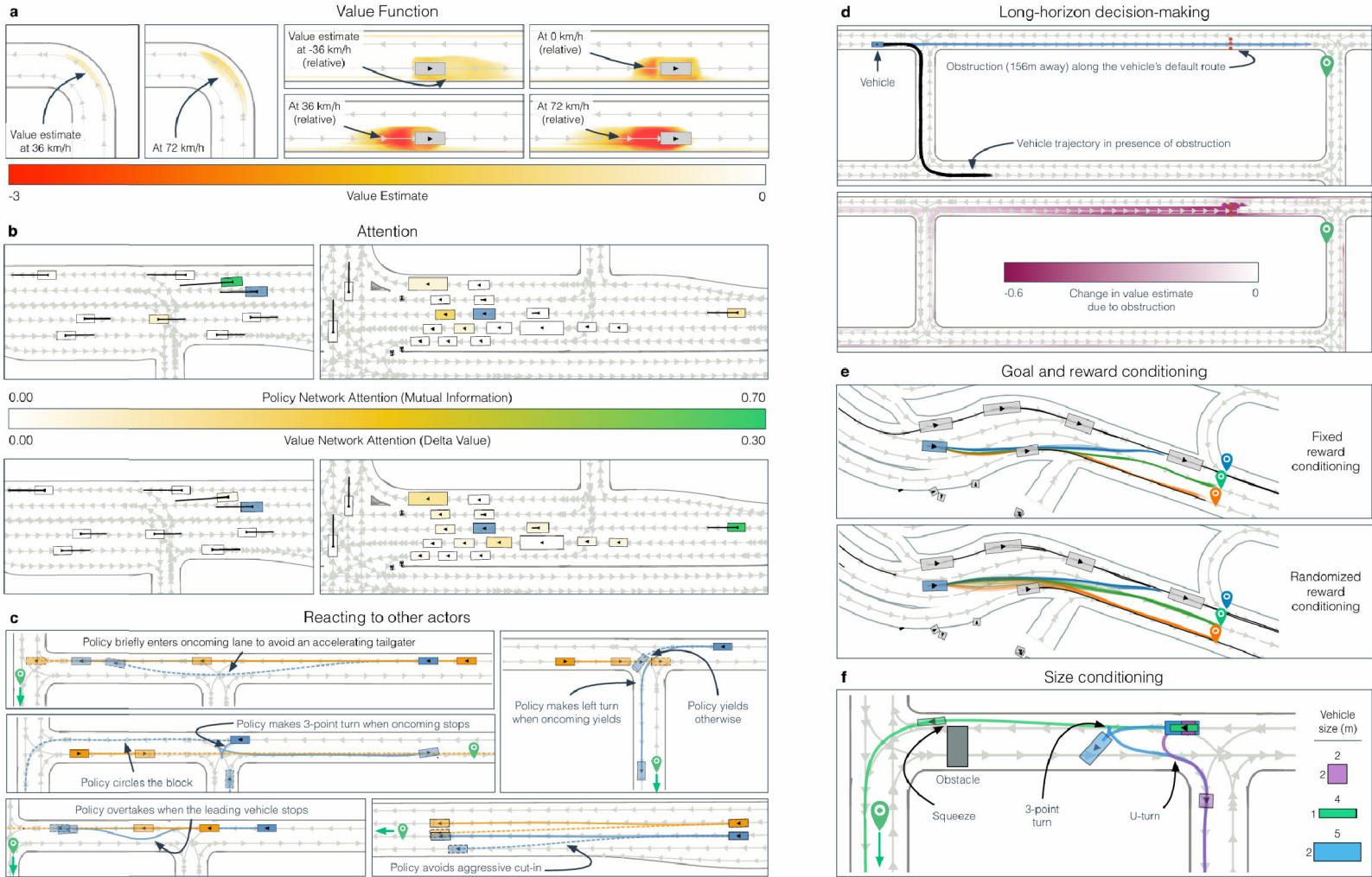
数据集上多个考察维度、都达到了该数据集历史最优成绩

模拟测试结果

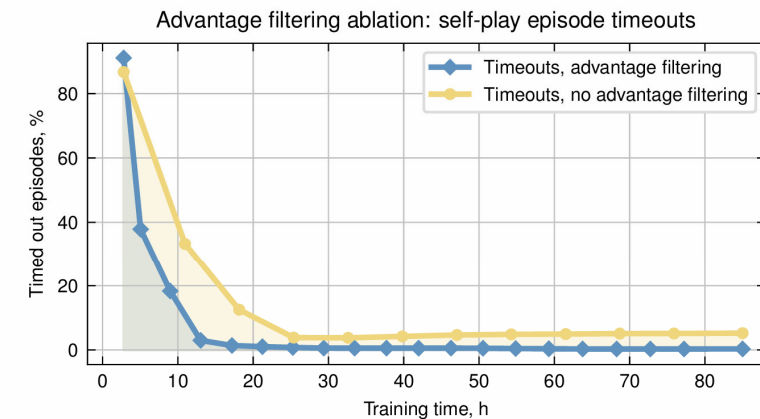
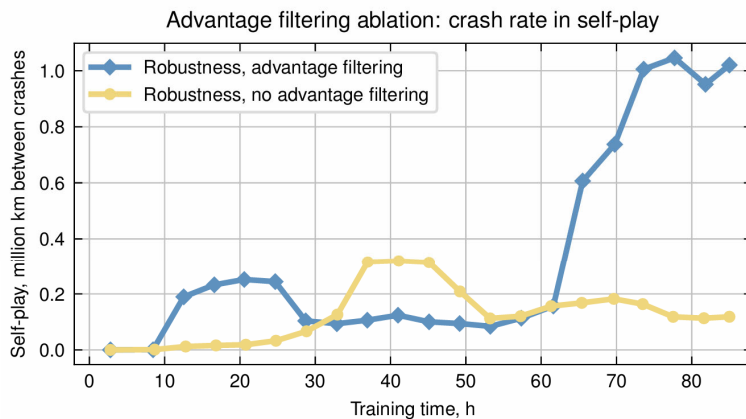
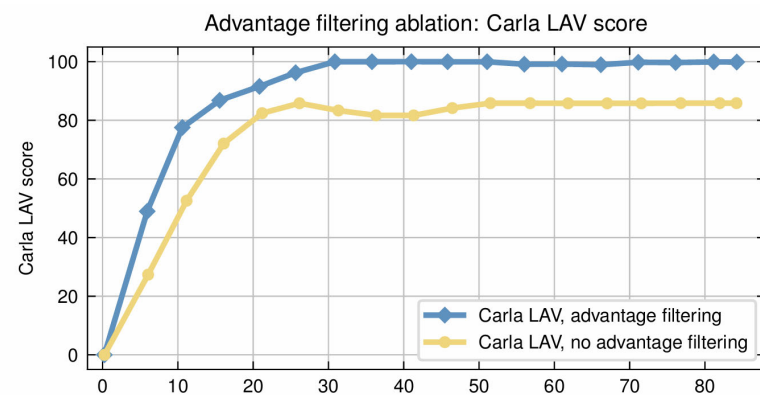
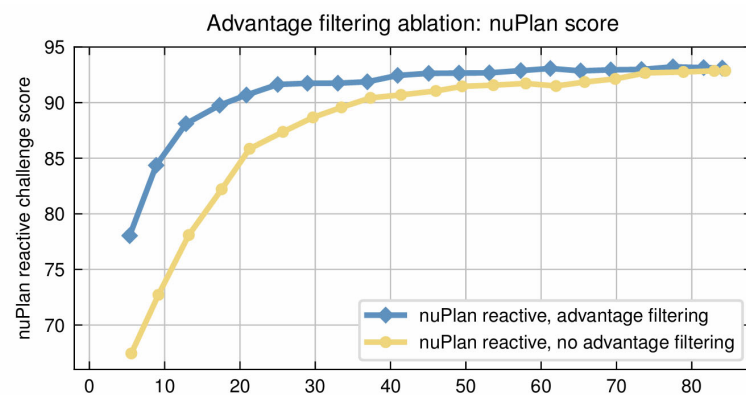


随着训练次数K增加，逐渐学会路口复杂并道、跟车、超车等操作

模拟测试结果



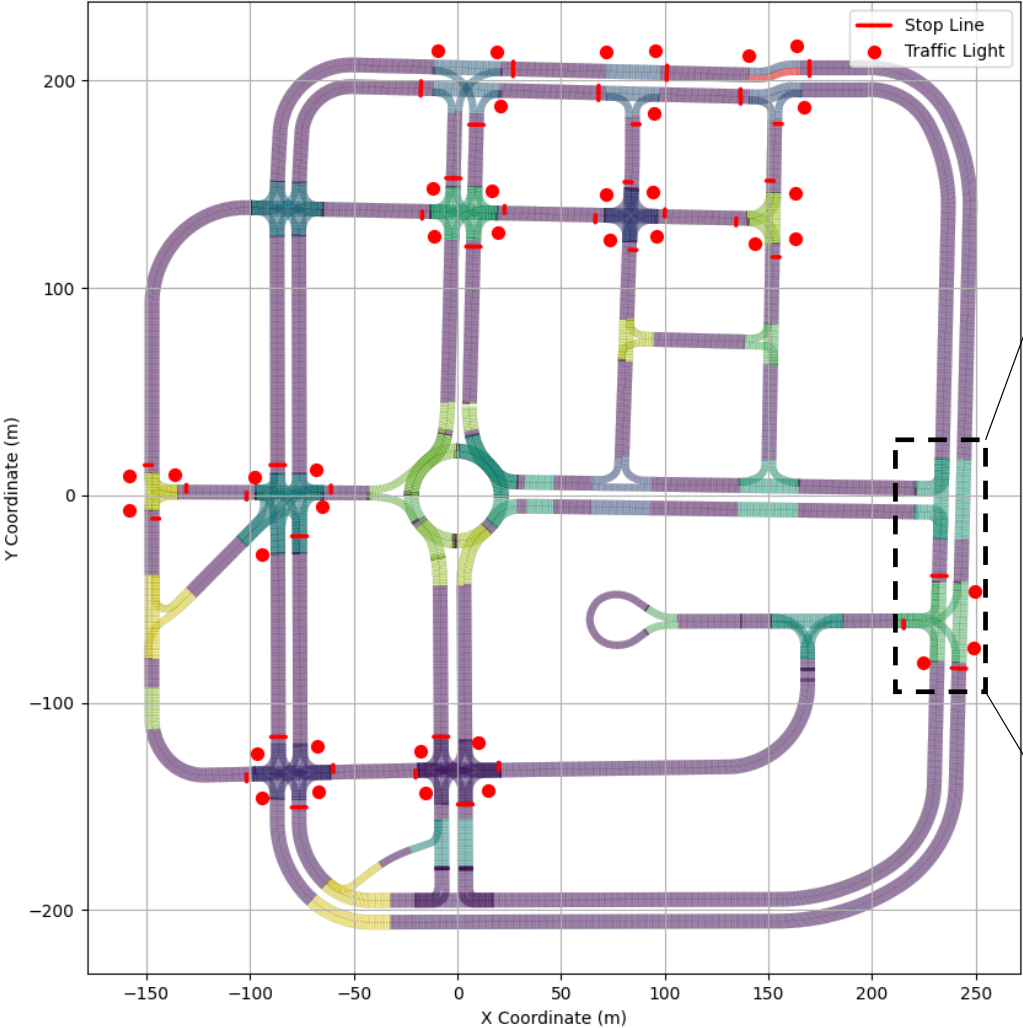
随着训练次数K增加，展现出惊人的与人类驾驶习惯一致性的现象



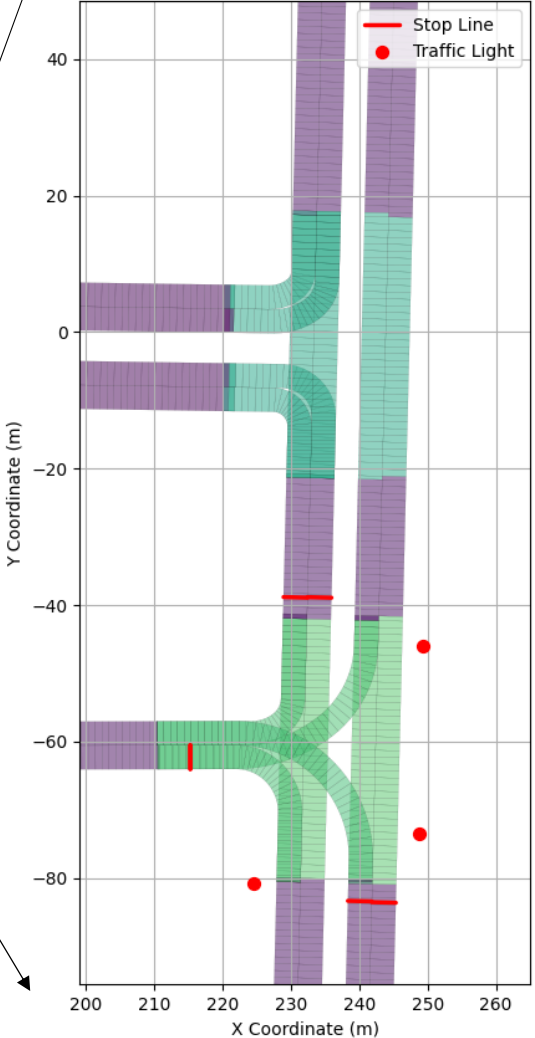
训练到80小时，已经做到平均100万公里才发生1次交通事故

World_init

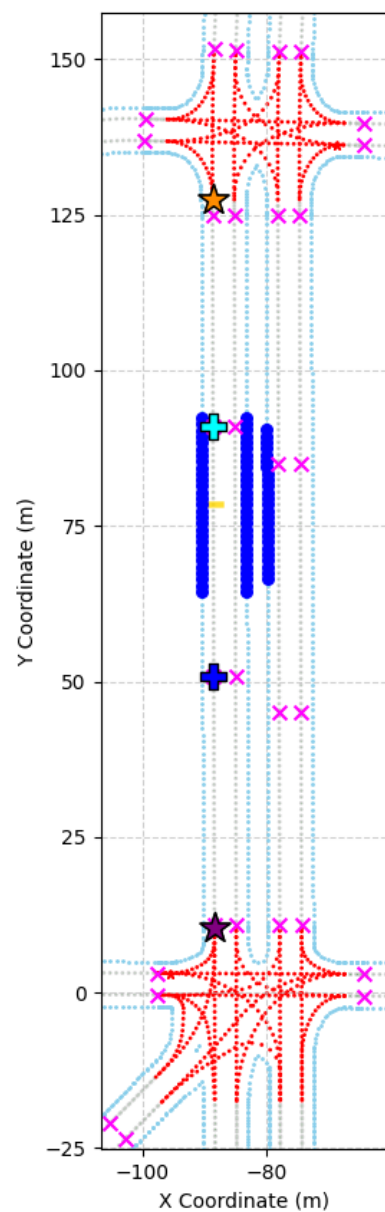
Map Visualization: Town03



Map Visualization: Town03



- quads
- poly_Id
 - vertices
 - road_id
 - lane_id
 - q
- traffic_controls
- traffic_light_id
 - traffic_light_location
- stop_line_waypoints
- location
 - rotation

World_init

cross_quad_center



all_quad_centers

$$[x,y,z]$$

oob_points


$$[x,y,z]$$


(partial observed)

global_w_lane_waypoints

 $[x,y,z]$

direction

routing

poly_id

carla_waypoint_info

id

road id

lane id

S

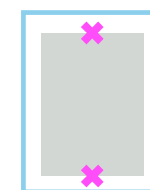
is_junction

~~next_poi_id~~~~dist_to_next_poi~~

```
prev_poi_id
```

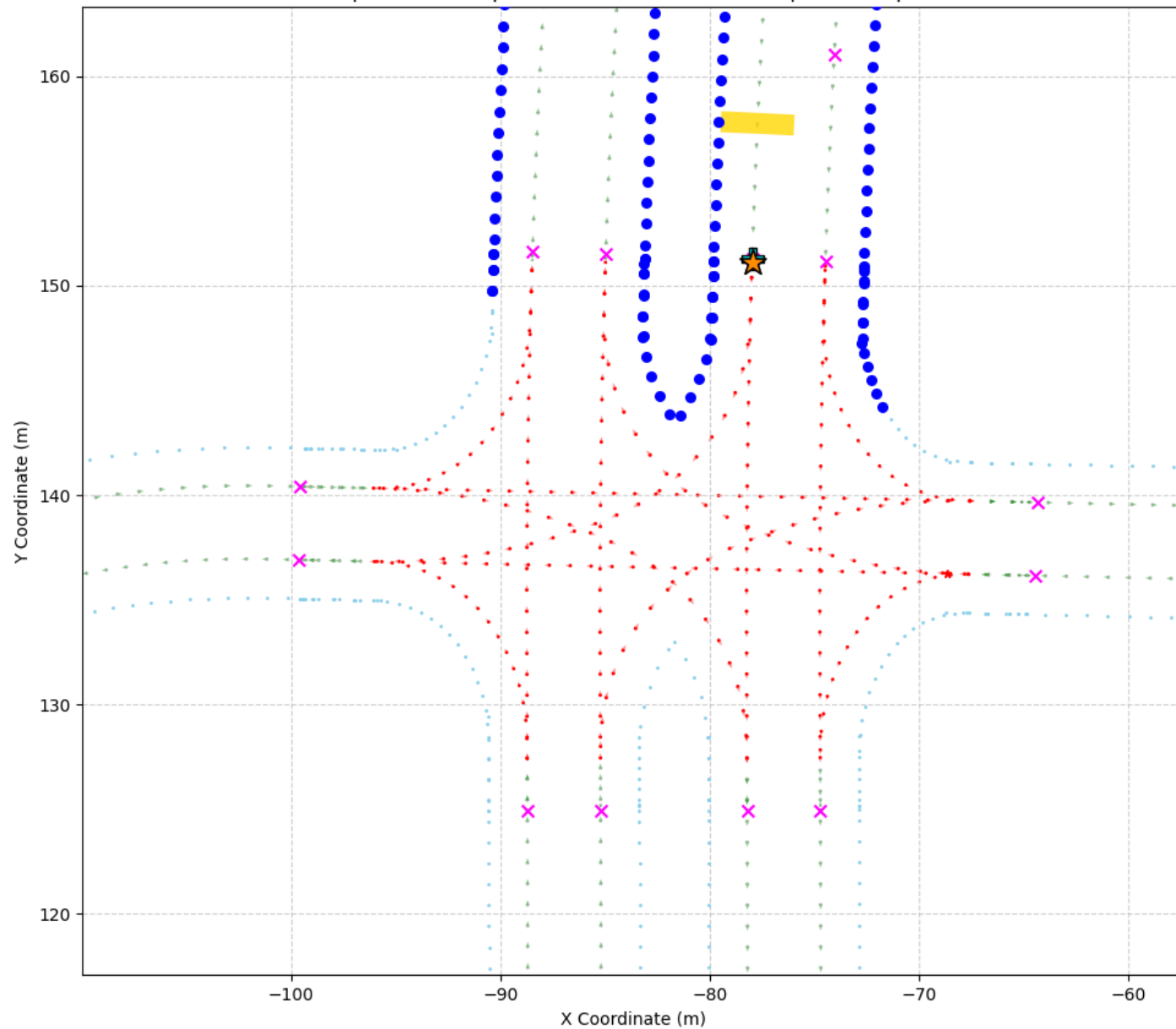
~~dist_to_prev_poi~~

waypoint_id

~~points_of_interest~~

oob points
0.1m away
from quads

Preprocessed Map Data Visualization (Click a quad to inspect)



World_init

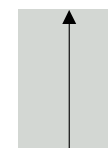
每个块的方向向量:

vertices[1]

P1

vertices[2]

P0



Extract vertices

P2

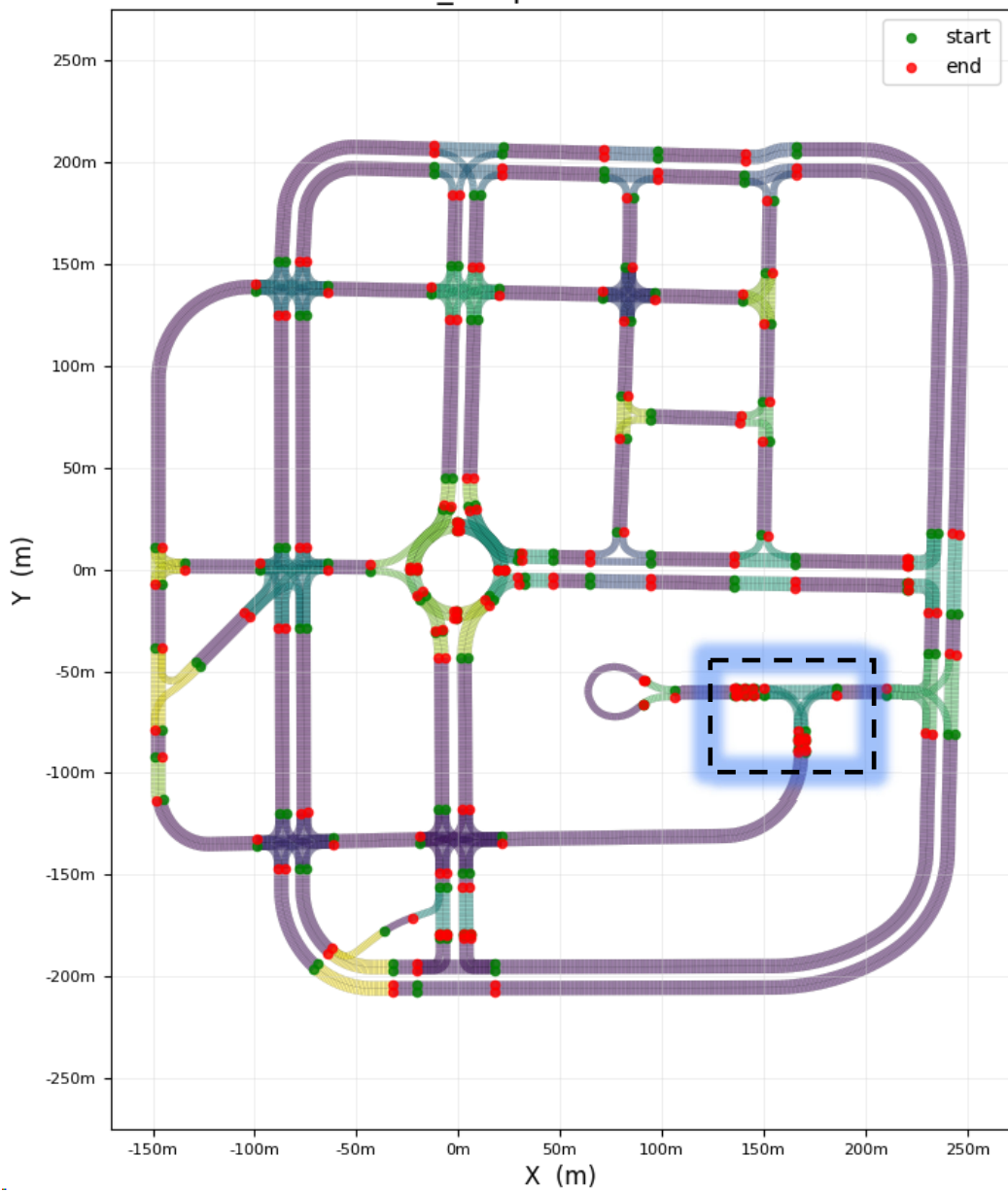
vertices[0]

P3

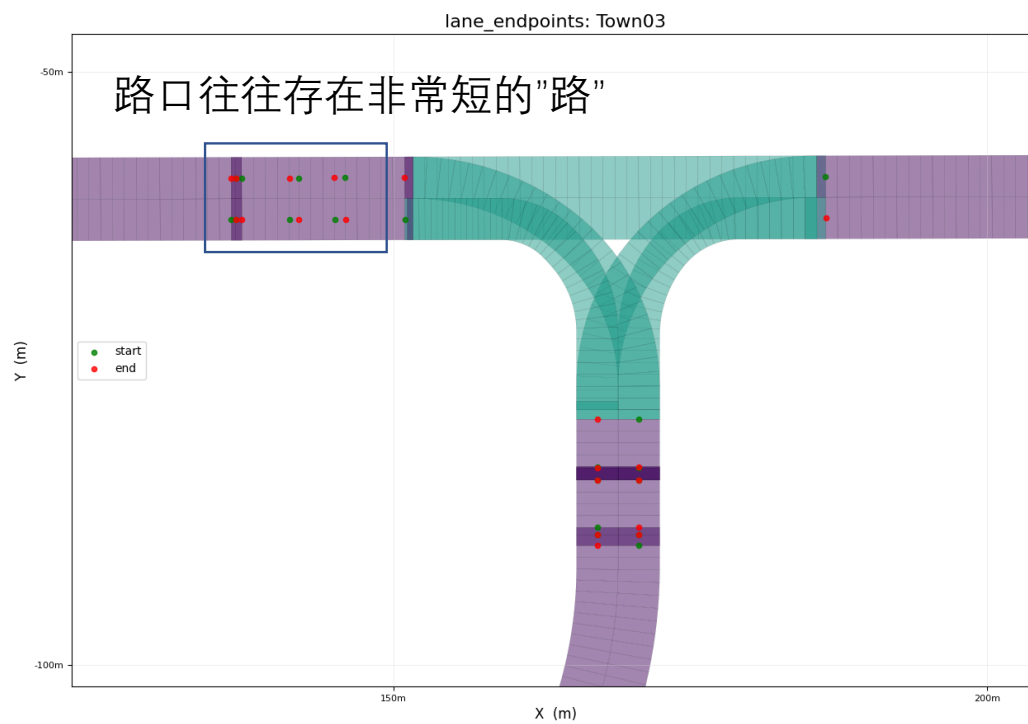
vertices[3]

朴素的向量

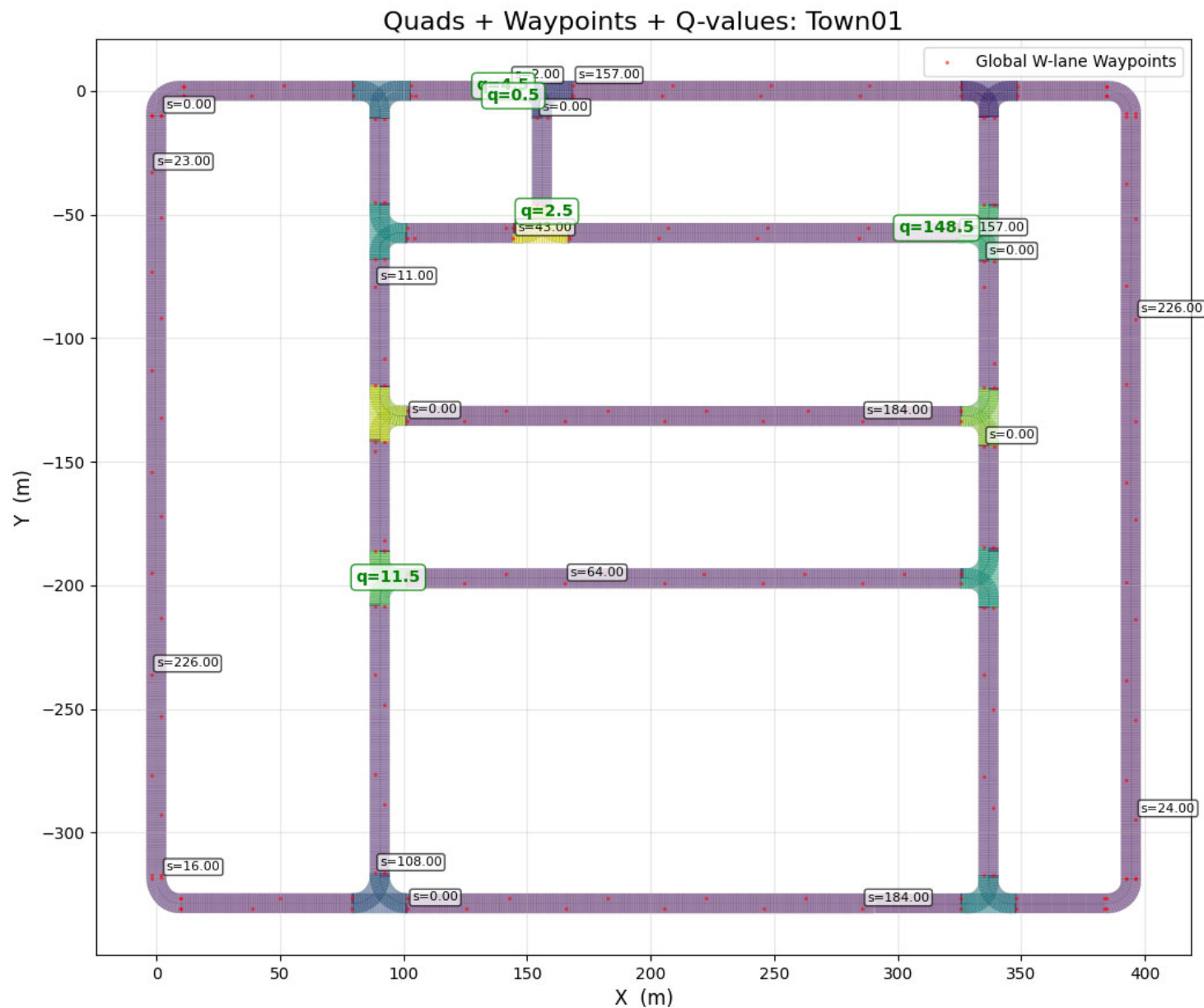
lane_endpoints: Town03



Lane endpoints



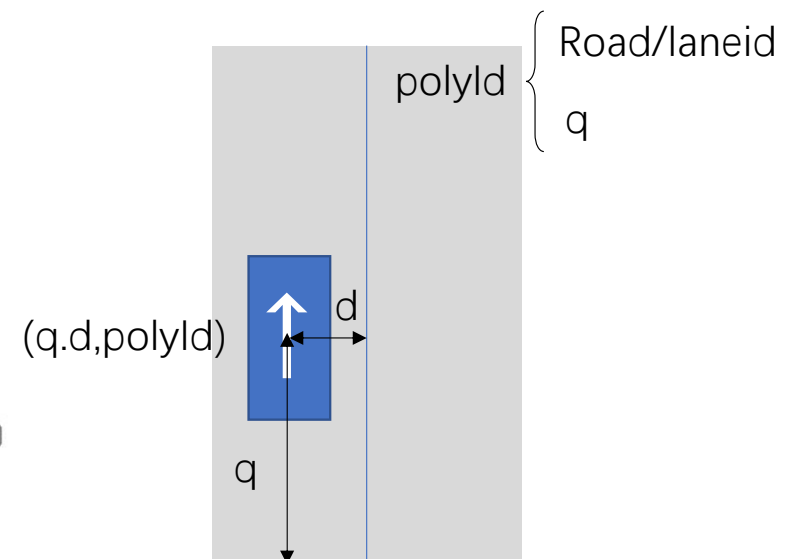
Frenet



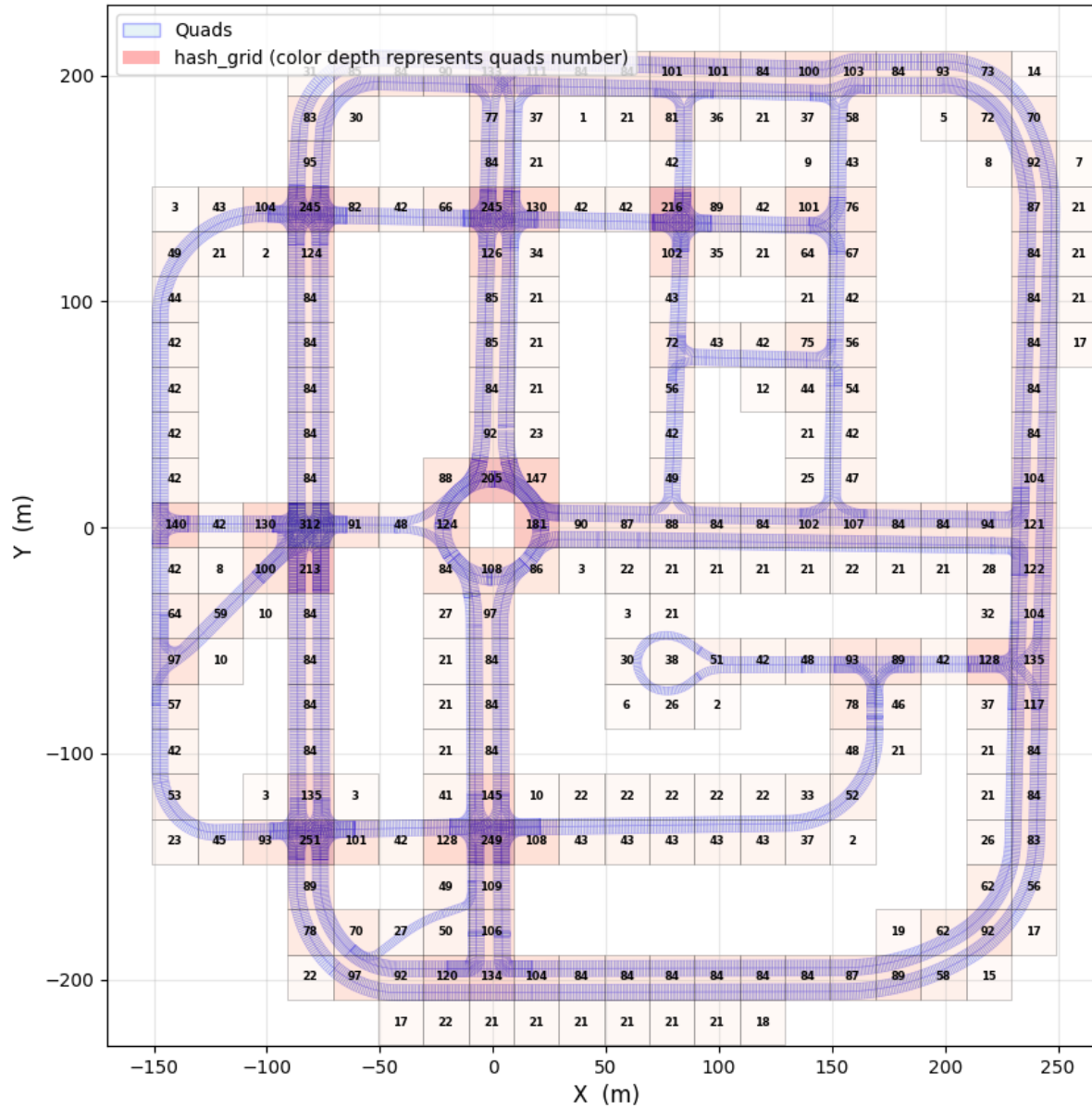
Driving waypoints的wp1.s, wp2.s均值
得到quads的q值(绿色)

而现在图上看到的Waypoints
已经是enhance过的。显得稀疏。

车辆在地图中坐标系:



Spatial Hash



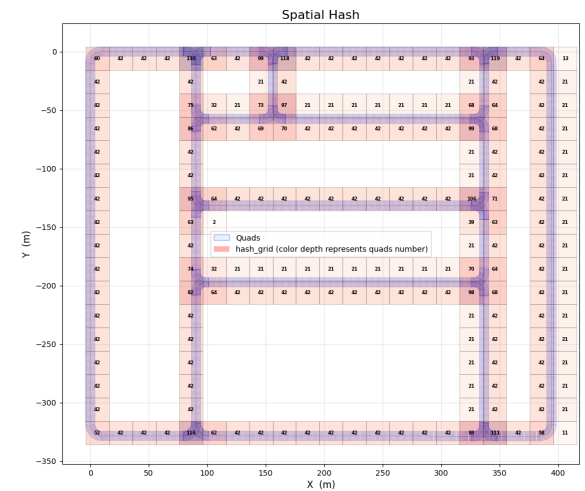
Spacial hash

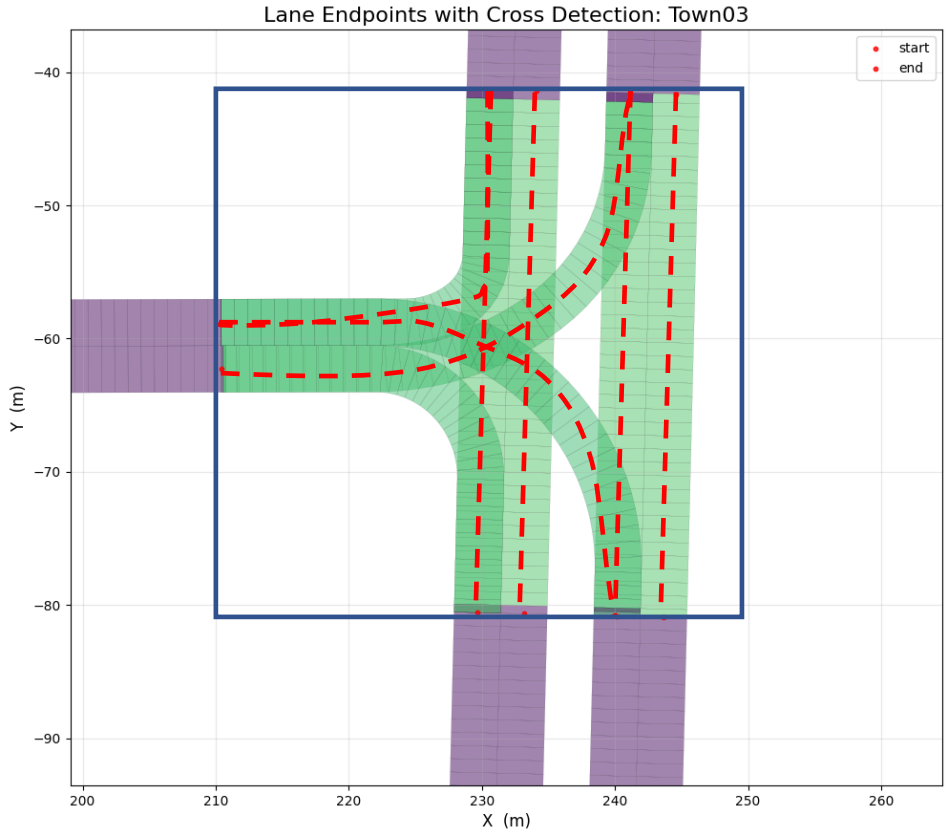
20mx20m

Bucket 两倍最大可能车长

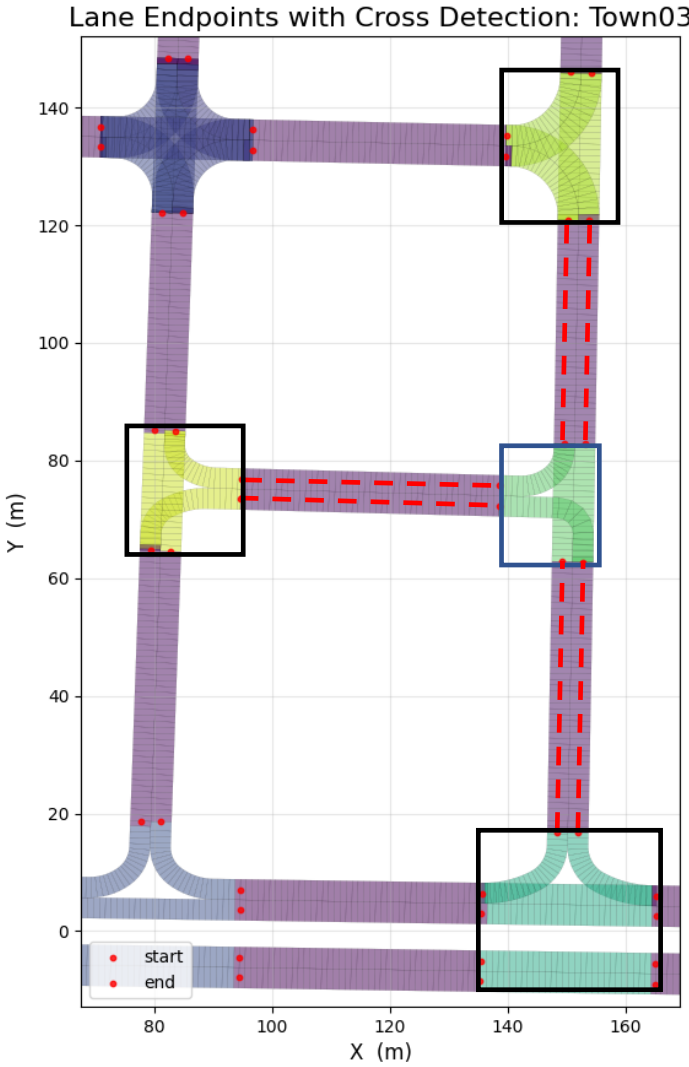
全局定位某个车的坐标
(x,y,mapId)

知道x,y就能定位bucket
也能知道所在的quad



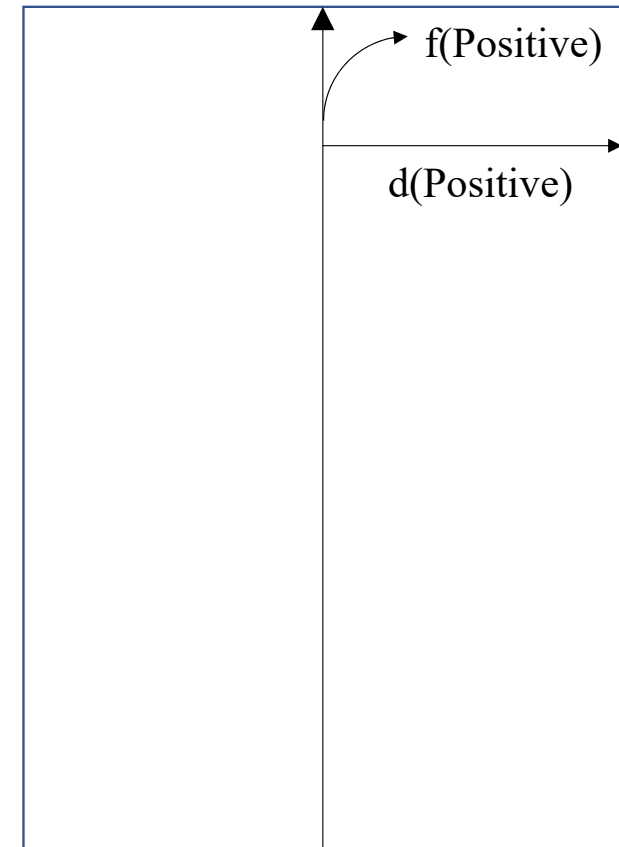
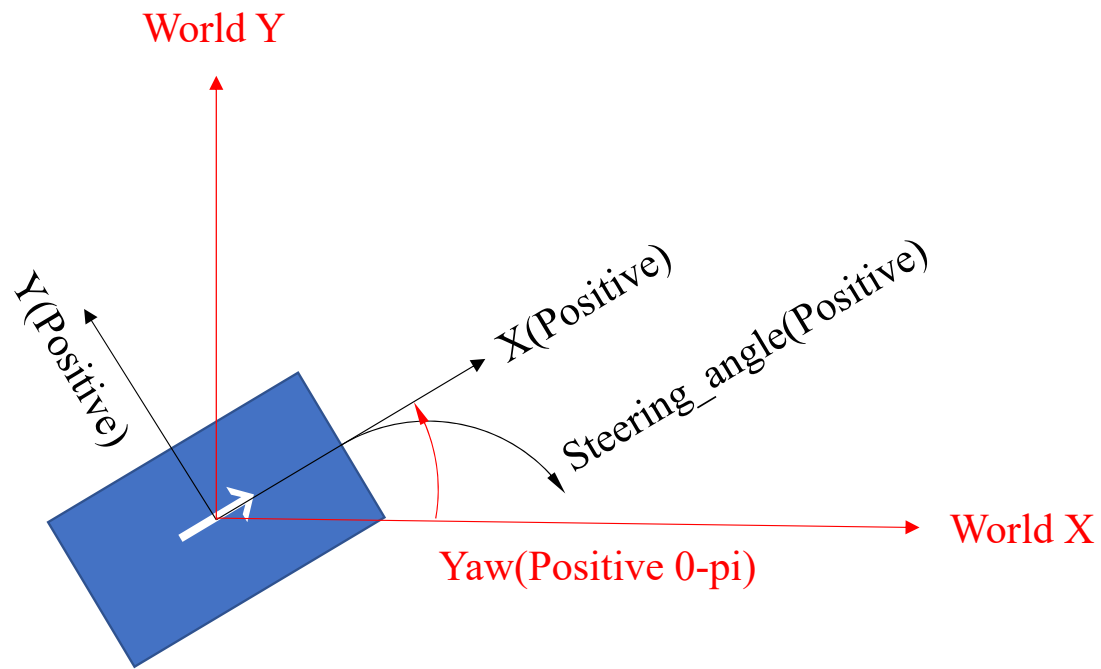


路口内的dijkstra路径储存

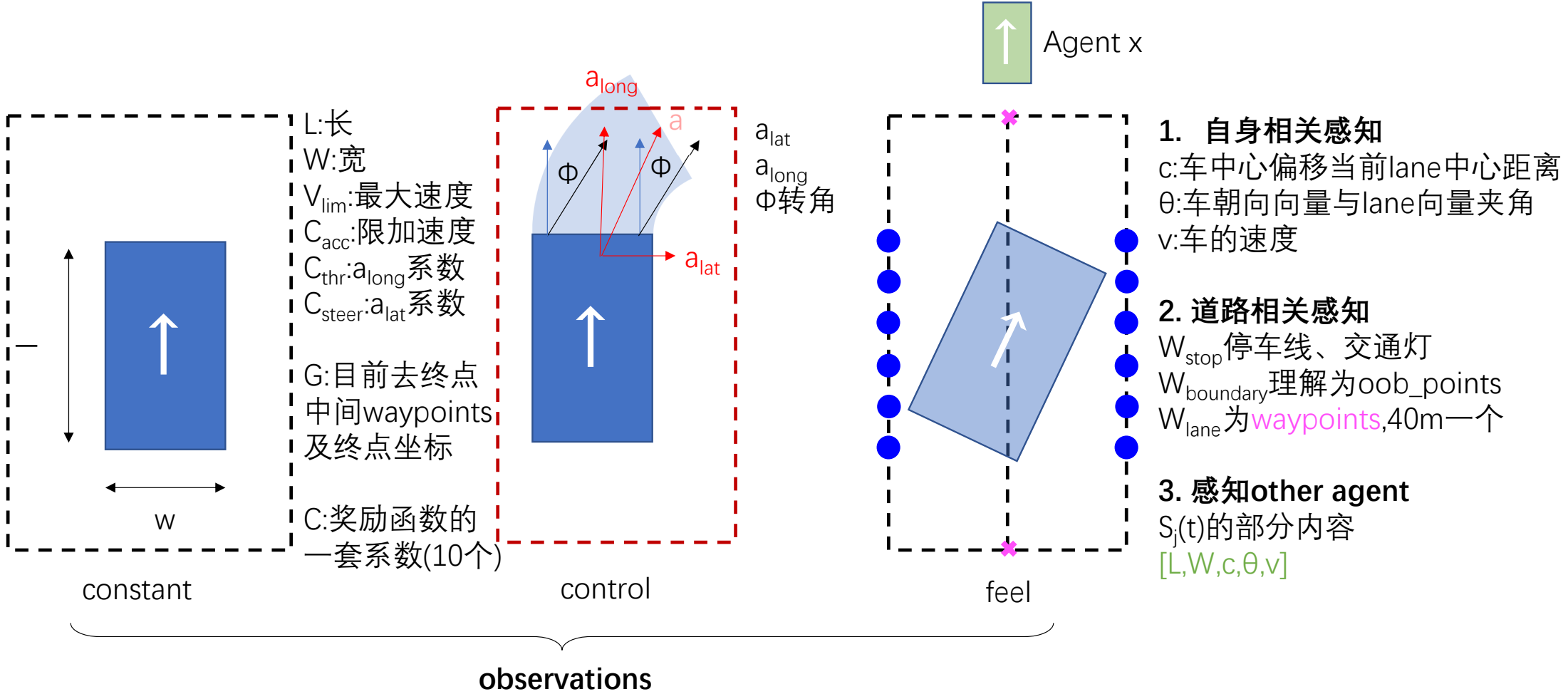


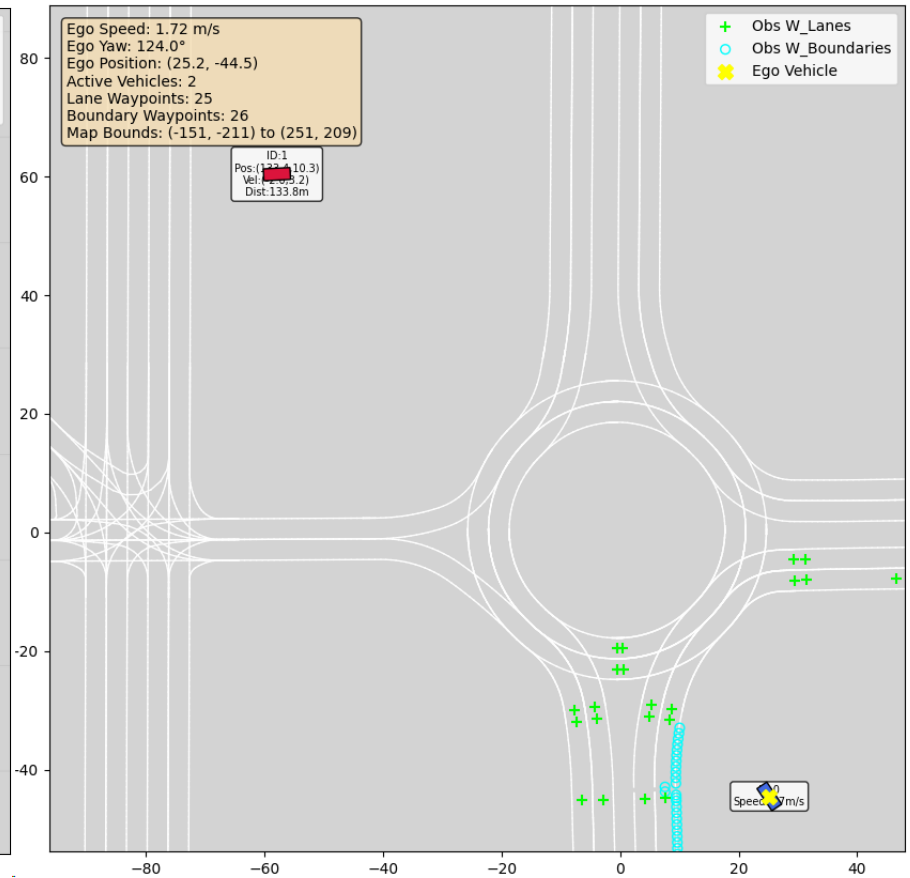
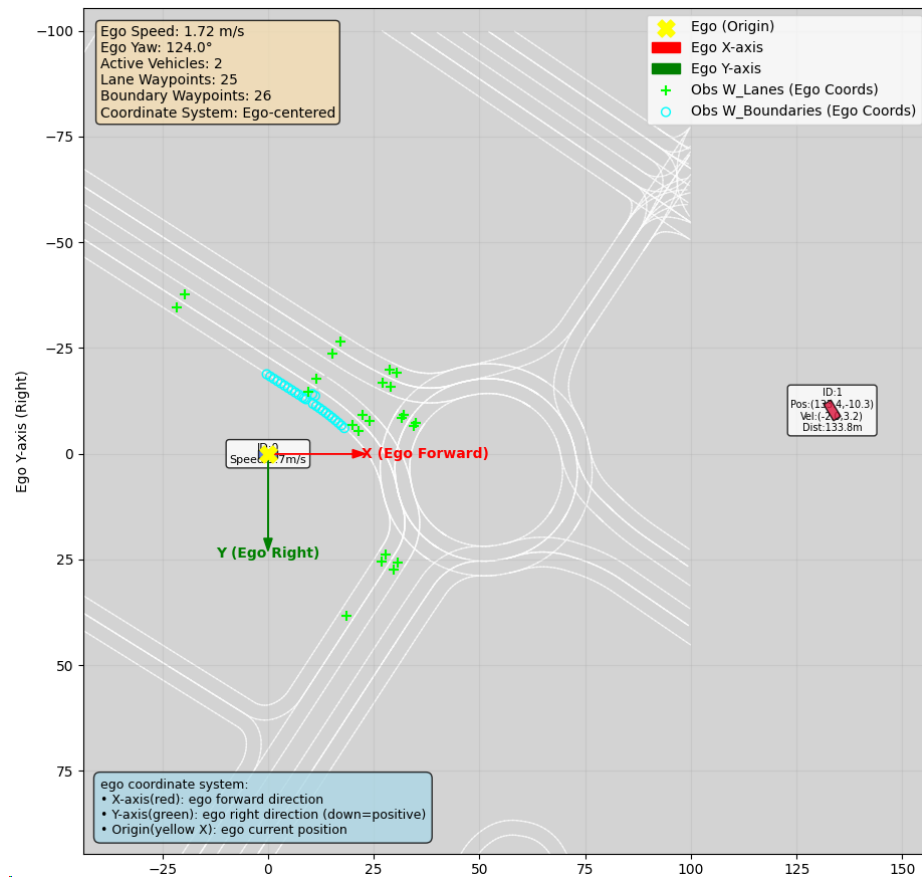
路外的dijkstra路径储存

Direction and coordinate specifications

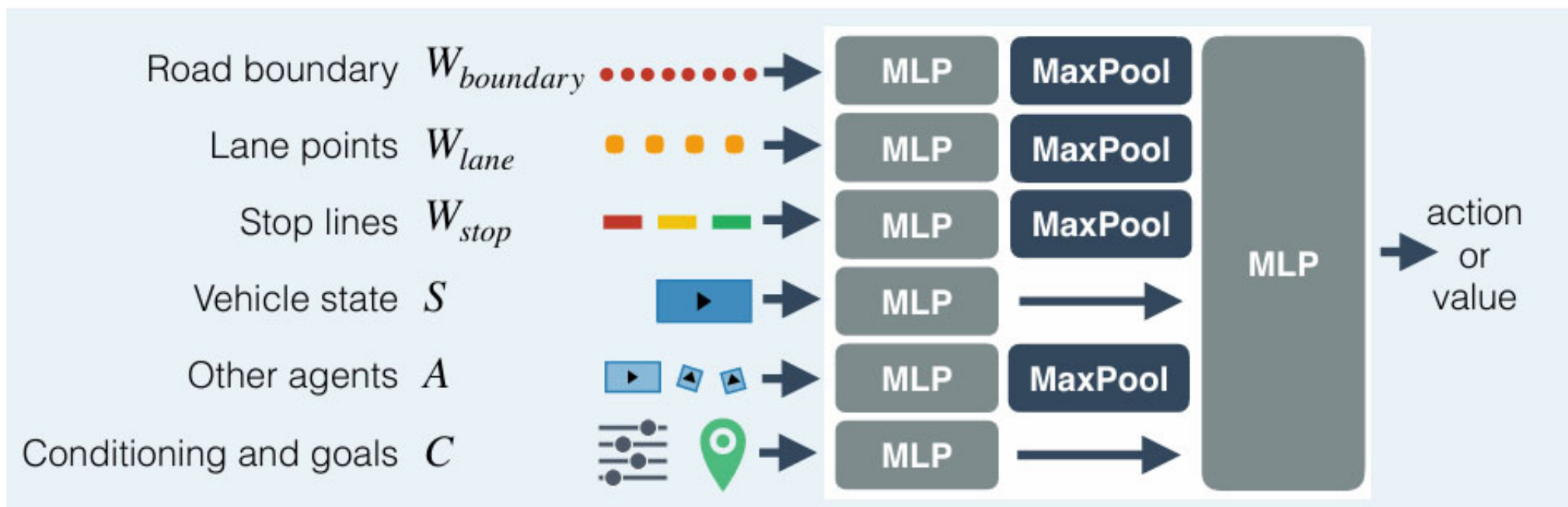


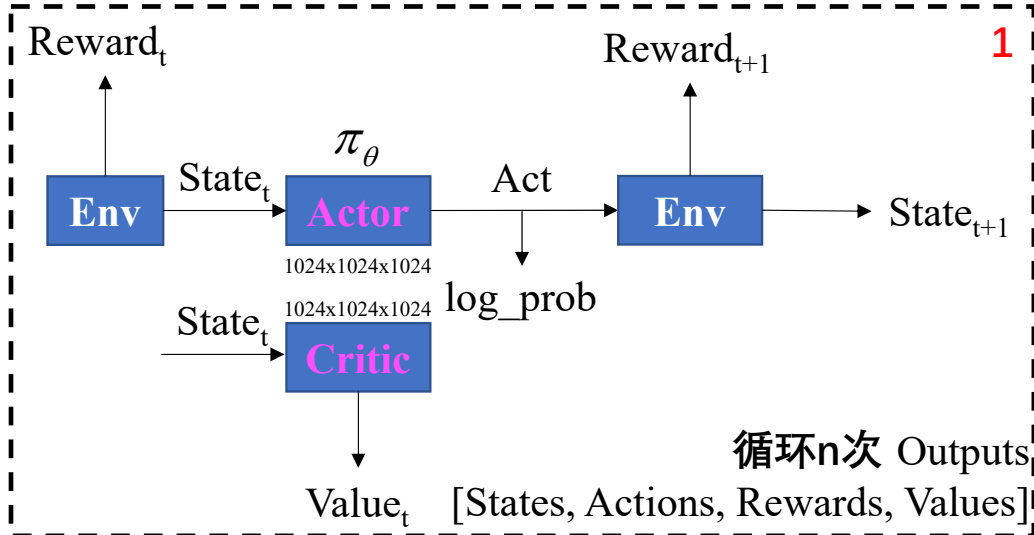
注：所有的观测量都归一化到 $[-1,1]$





动作索引	long纵向jerk (m/s ³)	lat横向jerk (m/s ³)	描述
0	-15.0	-4.0	急减速 + 急左转
1	-15.0	0.0	急减速 + 直行
2	-15.0	4.0	急减速 + 急右转
3	-4.0	-4.0	减速 + 左转
4	-4.0	0.0	减速 + 直行
5	-4.0	4.0	减速 + 右转
6	0.0	-4.0	匀速 + 左转
7	0.0	0.0	匀速 + 直行
8	0.0	4.0	匀速 + 右转
9	4.0	-4.0	加速 + 左转
10	4.0	0.0	加速 + 直行
11	4.0	4.0	加速 + 右转





GAE优势估计

$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t-1}\delta_{T-1}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$Q(s_t, a_t)$$

$$L_{\text{CLIP}}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \right) \right]$$

$$L_{\text{CLIP+VF+S}}^t(\theta) = \hat{E}_t \left[L_{\text{CLIP}}^t(\theta) - c_1 L_{\text{VF}}^t(\theta) + c_2 S[\pi_\theta](s_t) \right]$$

Adam优化器, optimizer.step()#更新参数

3

采样minibatch
states_batch
actions_batch
advantages_batch
 $Q(s_t, a_t)$ _batch

actions_batch $\rightarrow$ new_log_probs $\rightarrow r_t(\theta)$

4



$$L^{PolicyGradient} \longrightarrow L^{Clip}$$

$$L^{Clip} = \mathbb{E}[\min(r_t(\theta)A_t, clip(1 - \epsilon, r_t(\theta), 1 + \epsilon))]$$

$$L_t^{VF}(\theta) = (V_\theta(S_t) - V_t^{targ})^2$$

$$S[\pi_\theta](s_t) = -\sum \pi(a|s) \cdot \log(\pi(a|s))$$

$$[L^{Clip+VF+S} = \mathbb{E}[L^{Clip} + c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]]$$

反向传播、Adam更新网络参数

原始强化学习的目标函数：

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad \gamma \text{折扣因子, } r \text{是时间} t \text{步的奖励}$$

↓
最大化J等价于最大化策略生成的策略轨迹的概率

$$P(\tau | \theta) = \prod_{t=0}^{T-1} \pi(a_t | s_t, \theta)$$

$$\log P(\tau | \theta) = \sum_{t=0}^{T-1} \log \pi(a_t | s_t, \theta)$$

对于目标函数关于网络参数θ求偏导

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \ln \pi(A | S, \theta) Q_\pi(S, A)]$$

↓
▲ 实际使用中Q_π(S, A)波动大

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \ln \pi(A | S, \theta) (Q_\pi(S, A) - V(S))]$$

基于策略梯度
目标函数

优势函数A:
动作a的奖励和平均奖励值进行对比

$$L^{PolicyGradient} = \mathbb{E}[\ln \pi(A | S, \theta) A_t]$$

路縫隙问题

聚类 4 内找到 7 个起点和 7 个终点
无法找到从 13624 到 13659 的路径
无法找到从 13624 到 13899 的路径
无法找到从 13624 到 13902 的路径
无法找到从 13624 到 14119 的路径
无法找到从 13624 到 14121 的路径
无法找到从 13624 到 13618 的路径
无法找到从 13624 到 13620 的路径

_replication_maps(debug_quad_13624.py)

=== 分析Quad 13624的连通性问题 ===

总共有 14342 个quads

构建图...

✓ Quad 13624 在图中

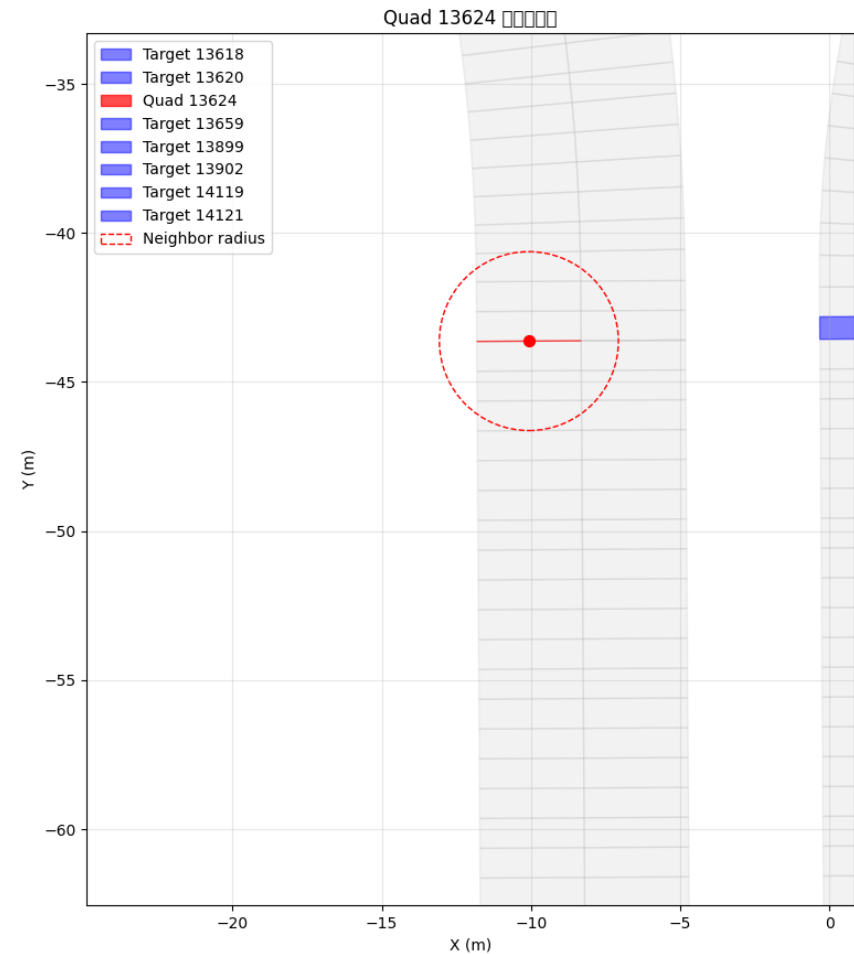
Quad 13624 的邻居数量: 0

✗ Quad 13624 是孤立节点!

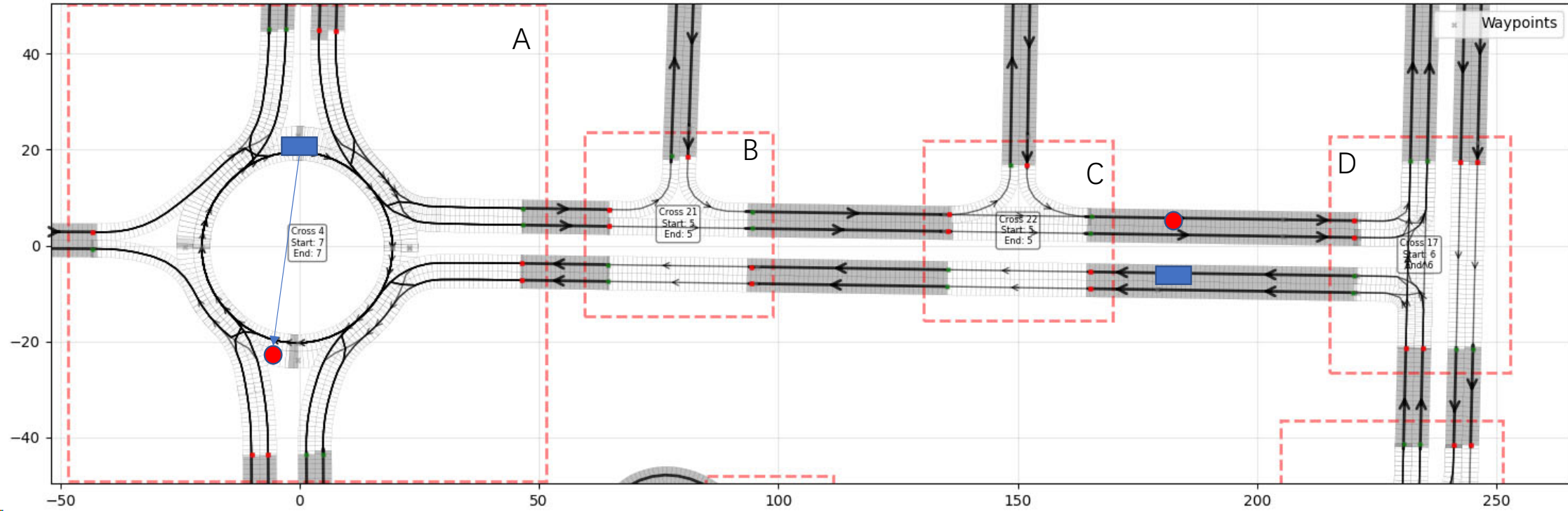
Quad 13624 中心位置: [-10.06715965 -43.62849045]

Quad 13624 方向向量: [0. 0.]

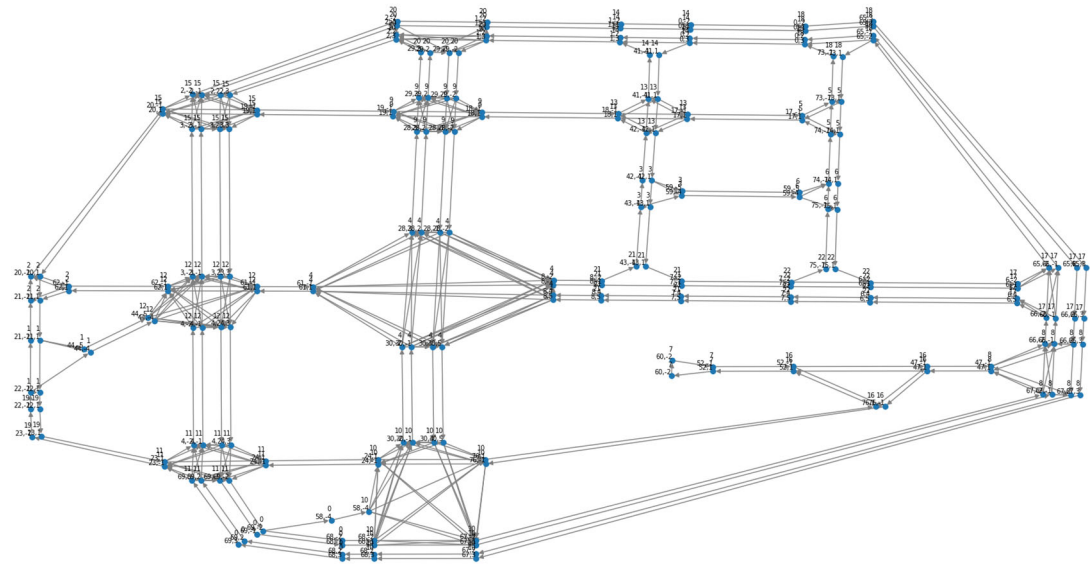
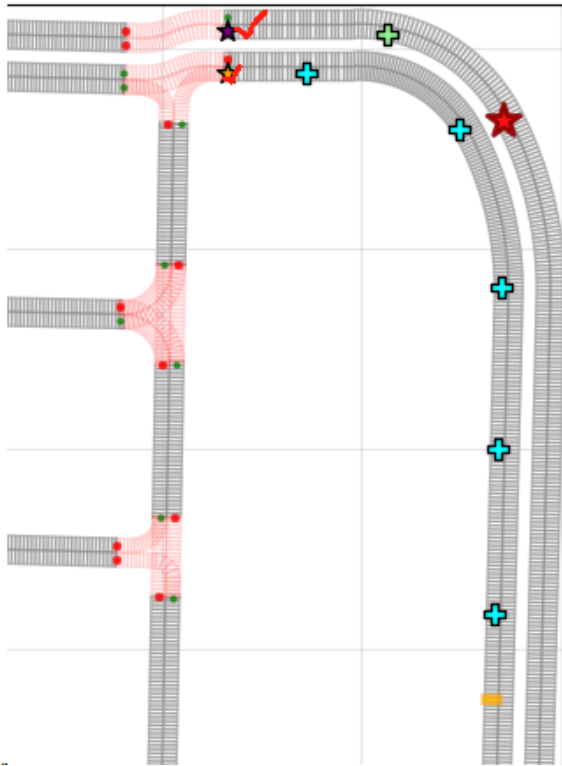
Quad 13624 方向向量长度: 0.0



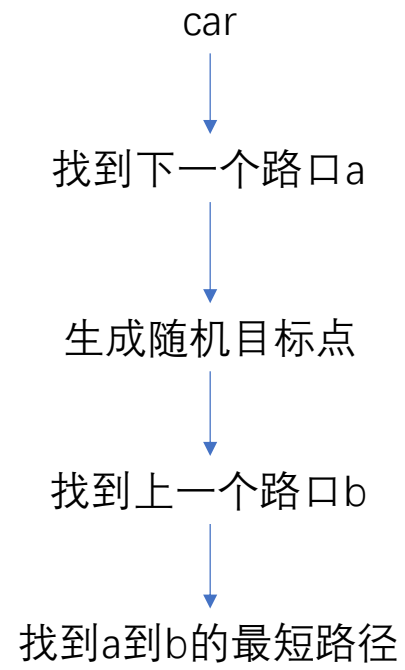
visualize_lane_paths_with_dijkstra - Town03



- 10wn03



问题转换为[cross_id, road_id, lane_id]到另一个的路径



91.7M→

cross_data_processed_map_Town01_stitched.json

filtered_quad_indices: 所有定义在路口之外的quad

cross_x:

cross_id: 每个cross都定义一个唯一id

start_waypoints: 所有可以从cross出去的点

x

y

road_id

lane_id

s

end_waypoints:

x

y

road_id

lane_id

s

paths:

from_end_waypoint

to_start_waypoint

path_quad_ids

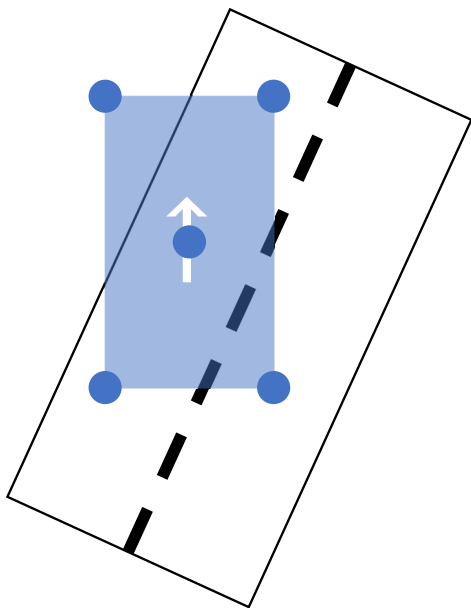
distance

waypoint_graph: 由路口所有的waypoint组成的路由

nodes:[cross_id,road_id,lane_id,x,y,"start"/"end"]

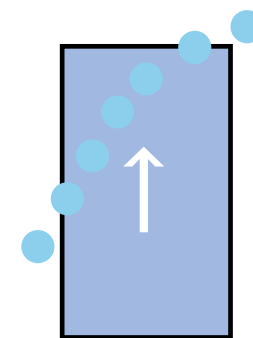
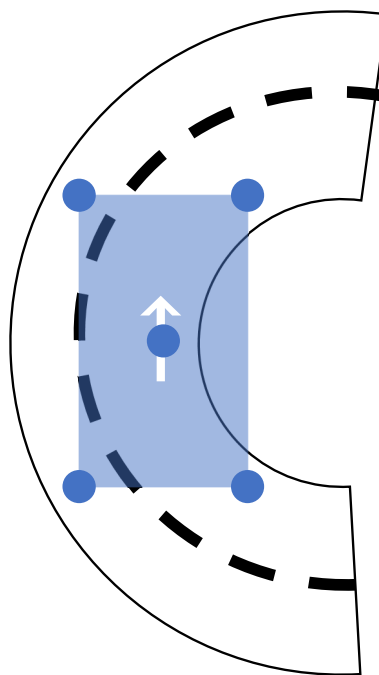
部分可观察的随机博弈 (POSG)

Basic Model



检查是否有点不在road surface上

弯路/环岛

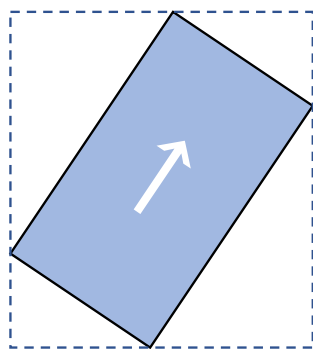
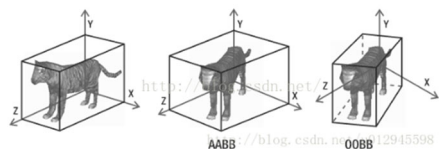


Bounding box

检测是否有OOB落到车框内

问题：oob太多

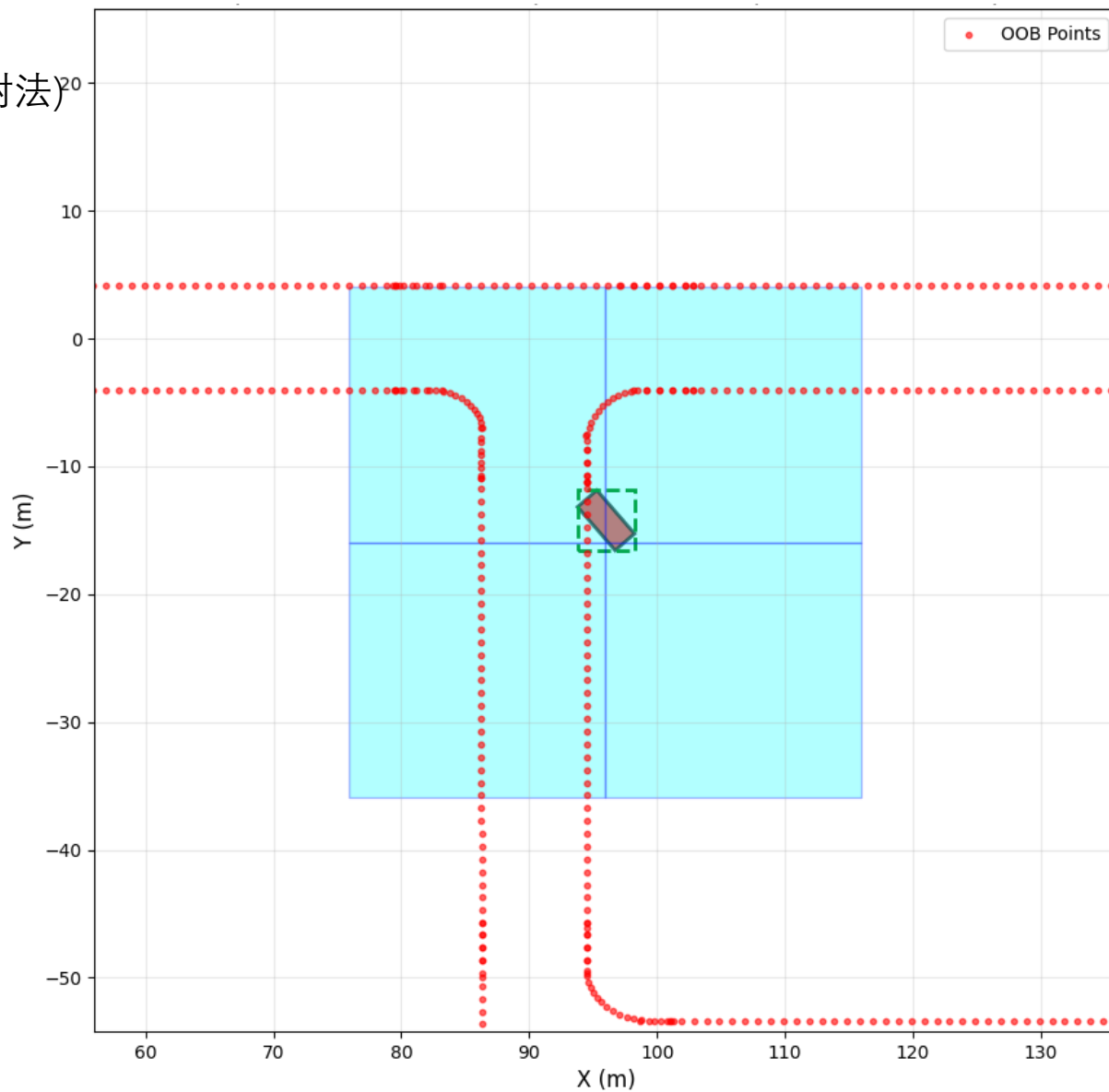
1. 找AABB涉及到的所有bucket的OOB点
2. 计算OOB是否在Bounding box内 (射线投射法)



AABB

由于bucket大小设计

最多AABB会跨越4个哈希bucket

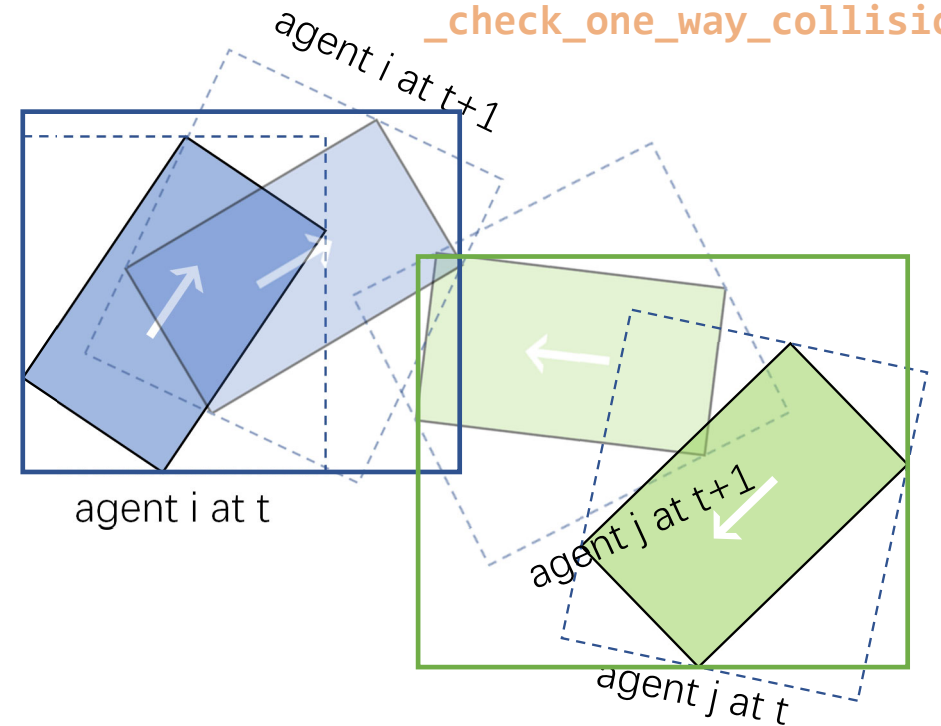
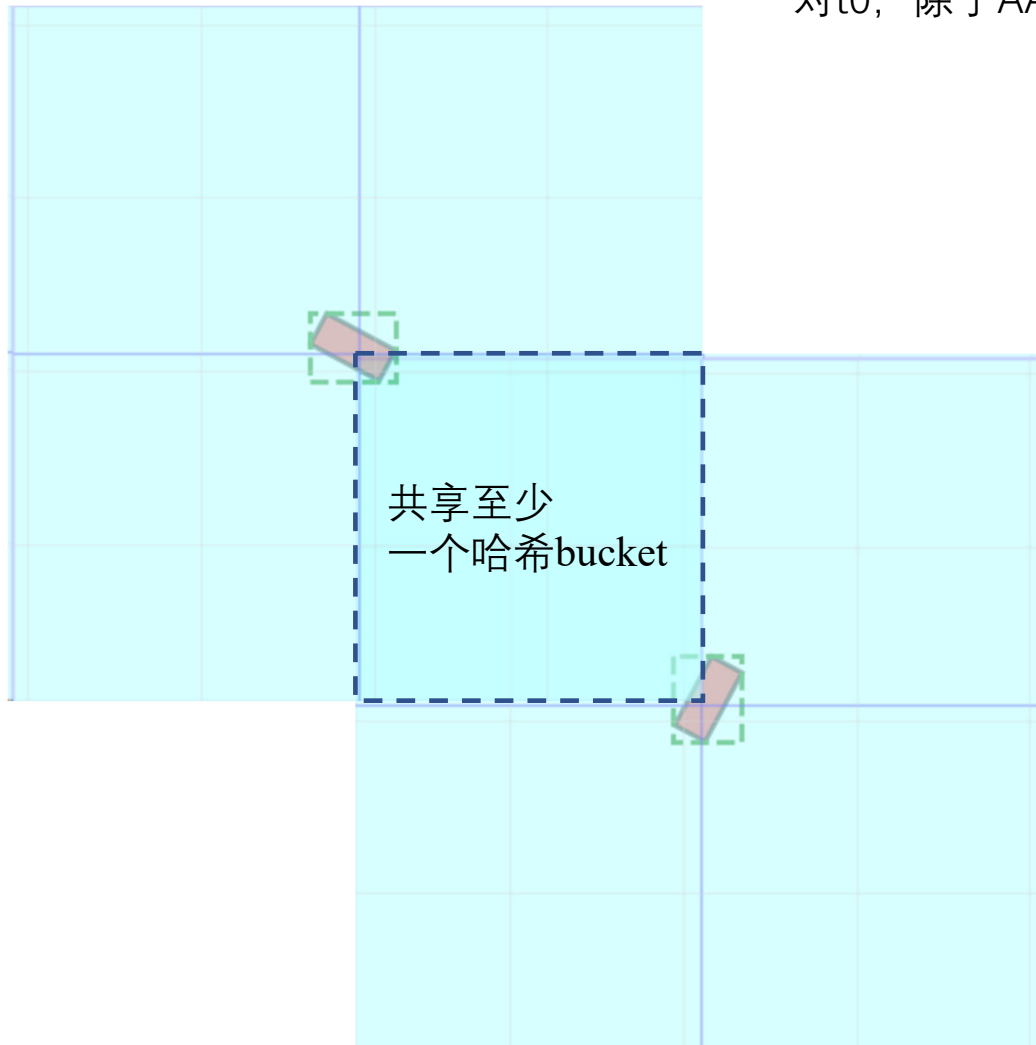


collision

对 t_0 ，除了AABB框是单独状态下的，其它碰撞检测和下面说的一样。

AABB框是同时包含 t 和 $t+1$ 的

_check_one_way_collision



相互检查 t 和 $t+1$ 时刻自己corners是否在对应时刻对方bounding box内

只要有一个检测撞了就是撞了

在二维平面上检测到碰撞后，进一步检查这些碰撞是否在三维空间中实际发生。

Setting/Parameter	Value
Vehicle length	$l \sim U(2, 5.5)$
Vehicle width	$w \sim U(1.5, 2.5)$
Maximum speed	20 m s^{-1}
Timestep Δt	0.066 s
Episode length	9000 steps (600 s)
Agents per sim N_a	50
Observed agents N_o	up to 40 closest
δ_{goal}	10 m
v_{goal}	3 m s^{-1}
$\alpha_{\text{collision}}$	3.0
α_{boundary}	3.0
α_{comfort}	0.05
$\alpha_{\text{l-align}}$	2.5×10^{-2}
$\alpha_{\text{vel-align}}$	1.0
$\alpha_{\text{l-center}}$	3.8×10^{-3}
$\alpha_{\text{center-bias}}$	0.0
α_{velocity}	2.5×10^{-3}
α_{reverse}	5.0×10^{-3}
$\alpha_{\text{stop-line}}$	1.0
α_{timestep}	2.5×10^{-5}

Table A4: GIGAFLow settings for long-form evaluation in self-play.

observation

agents_state: $B \times M \times 7$ $\longrightarrow$ **generate** $\longrightarrow$ observation: $B \times M \times 151$

$B=38400$ #世界数量
 $M=1-150$ #最多的agent数目

agents_state:
[x, y, yaw, speed, length, width, active]

总长度: $B \times M \times 7$

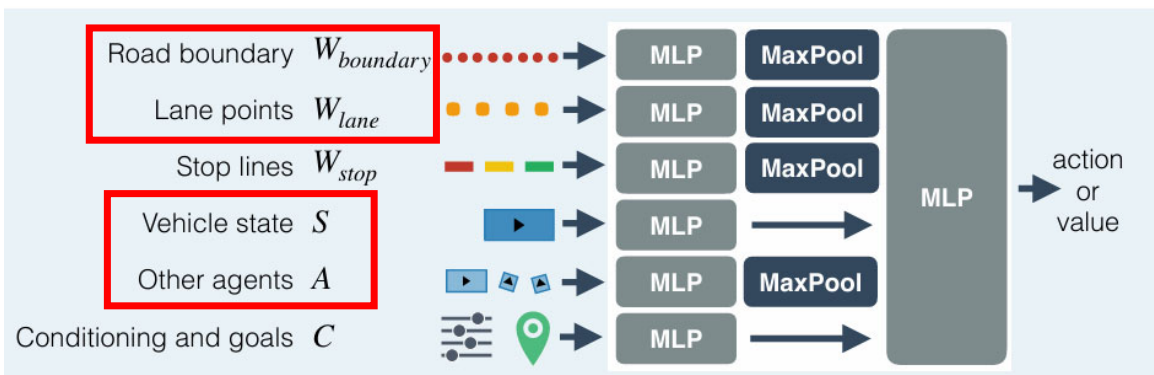
$B=38400$ #世界数量
 $M=1-150$ #最多的agent数目

[
local_state_dim:
[x, y, yaw, speed]:4

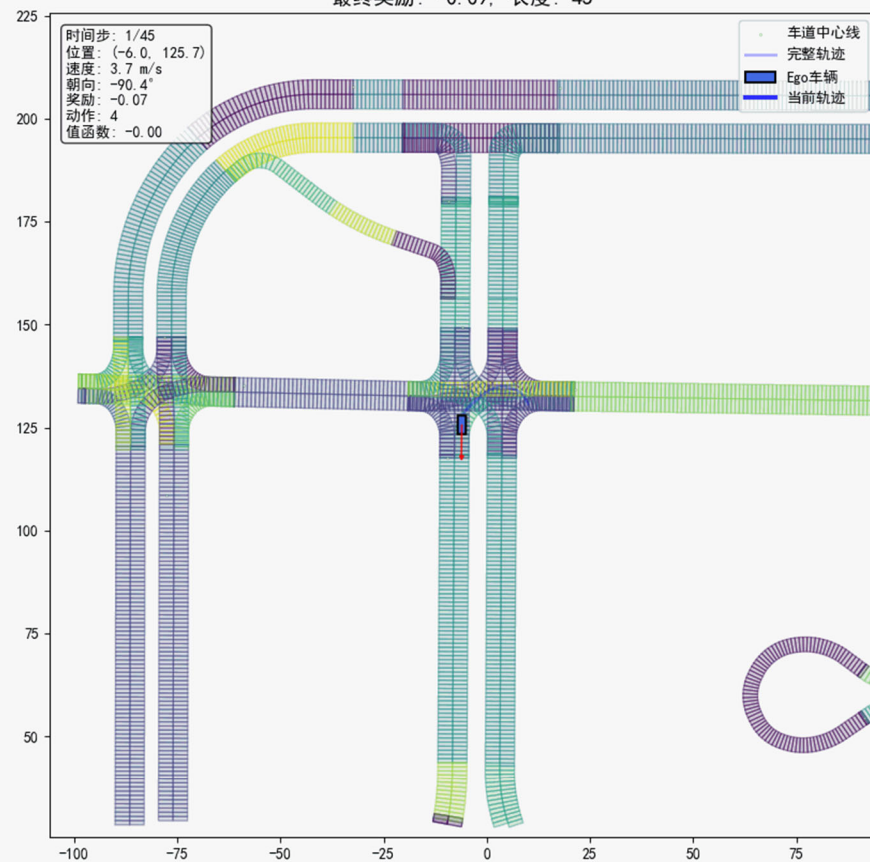
neighbor_feature_dim: 5
num_neighbors: $M-1$
[x, y, yaw, speed, activate]: $5M-5$

waypoint_feature_dim: 2
num_w_lanes: 25
num_w_boundaries: 26
[x,y,...,x,y]: 102
]

总长度: $B \times M \times (5M+101)$



Episode 20 轨迹
最终奖励: -0.09, 长度: 45



Episode 42 轨迹
最终奖励: -0.11, 长度: 40

